

Clinical trial eligibility screener: phased build plan

Drop this file in the repo root. Antigravity can read it, so every prompt below starts with "Read PHASES.md".

Phases 0 to 7 build what the problem statement asks for. Do not start phase 8 until the checkpoint after phase 7 passes.

How to use this with Antigravity

- Run one phase per agent task. Small tasks fail less and are easier to review.
- Use Planning mode. Read the implementation plan Antigravity produces and correct it before you let it write code. That review is your "decide and correct" step, and the jury will ask about it.
- After each phase, run the "Done when" check yourself. Don't trust the agent's own summary.
- Use the browser agent to click through the React UI after frontend phases.
- Commit after every phase that passes: `git commit -m "phase N: ..."`. If a later phase

breaks things, roll back.

- Copy every prompt you send, plus any correction you made, into `docs/prompts.md`. This is evidence for "Did you use AI deliberately?"
- Edit the prompts to match your own decisions. Don't paste them blindly.

Project structure

```
screener/  
  PHASES.md  
  docs/          spec.md, tasks.md, api.md,  
prompts.md  
  backend/  
    app/  
      main.py      FastAPI app and  
routes  
      config.py     reads .env  
(LLM_BASE_URL, LLM_MODEL, LLM_API_KEY)  
      llm.py        one function that  
returns the LangChain chat model  
      models.py     Pydantic models  
      db.py         SQLite helpers  
      parser.py     criteria text to JSON  
rules  
      rules.py      rule engine  
      explainer.py  rationale generation  
      screening.py  ties parser, rules,  
explainer together  
      evaluate.py   accuracy metrics  
data/generate.py   synthetic data
```

```
generator
  tests/
  frontend/          React + Vite +
Tailwind + Recharts
```

Shared data models

Every phase uses these. Put them in `docs/spec.md` in phase 0.

- Trial: `id, title, condition, criteria_text, rules[]`
- Rule: `id, kind (inclusion|exclusion), field, operator, value, unit, source_text`
- Patient: `id, age, sex, conditions[], medications[], labs{hba1c, egfr, bmi, ...}, pregnant, notes`
- CriterionResult: `rule_id, status (MET|NOT_MET|UNKNOWN), patient_value, evaluated_by (rules|llm)`
- Verdict: `patient_id, trial_id, decision (ELIGIBLE|NOT_ELIGIBLE|NEEDS_REVIEW), results[], rationale, confidence`

Decision logic, in this order:

1. Any exclusion rule MET gives NOT_ELIGIBLE.
2. Any inclusion rule NOT_MET gives NOT_ELIGIBLE.
3. Any rule UNKNOWN gives NEEDS_REVIEW.

4. Otherwise ELIGIBLE.

Confidence is the share of rules with a known status (MET or NOT_MET).

The parser can only map criteria to a fixed list of field names. If a criterion doesn't fit any field, the parser marks it `field: "notes"` and the LLM checks it against the patient's notes. That keeps the decision deterministic wherever the data allows it.

Part A: required features

Phase 0: decompose, spec, setup (0:00 to 0:20, whole team)

First 10 minutes without laptops. Answer the five questions from the briefing, sketch the architecture, and agree on scope. Then:

```
Read PHASES.md. Using the "Shared data
models" and "Project structure"
sections, write docs/spec.md (scope, users,
core flow, data models,
decision logic, out of scope) and
docs/tasks.md (a checklist of phases
0 to 12). Then create the empty folder
structure, a Python venv with
fastapi, uvicorn, langchain, langchain-
```

```
openai, pydantic, scikit-learn,  
chromadb, and a requirements.txt. Create a  
.env.example with  
LLM_BASE_URL, LLM_MODEL, LLM_API_KEY. Do  
not write feature code yet.
```

Also write `docs/api.md` by hand with the JSON shapes for these endpoints. Frontend and backend both code against it.

- `GET /trials`
- `POST /trials`
- `GET /patients`
- `POST /screen` with body `{trial_id, patient_ids}` returning a list of Verdicts
- `GET /metrics`

Done when the folders exist, the venv installs, and the team agrees on `spec.md`.

Phase 1: synthetic data (0:20 to 0:35, P2)

```
Read PHASES.md and docs/spec.md. Write  
backend/data/generate.py that  
creates, with a fixed random seed:  
- 4 trials as JSON: type 2 diabetes,  
hypertension, chronic kidney  
disease, breast cancer. Each has 5 to 8  
inclusion/exclusion criteria
```

```
written as free text, like a real
protocol.
- 30 patients matching the Patient model,
with realistic labs and a
  short clinical note each.
Deliberately include edge cases: age
exactly on a boundary, a missing
lab value, a lab in mmol/L instead of
mg/dL, a note saying "no history
of myocardial infarction", a note that
contradicts a structured field,
and one implausible value (HbA1c 45).
Also write backend/data/labels.json with
the expected decision for
every patient-trial pair, and a short
comment explaining each label.
Save everything to backend/data/.
```

Check the labels by hand. The accuracy number is only as good as these labels. This is a good place for your own domain judgment.

Done when `python data/generate.py` writes trials, patients and labels, and you have checked at least 10 labels yourself.

Phase 2: backend skeleton and LLM connection (0:20 to 0:35, P4 and P1 in parallel with phase 1)

```
Read PHASES.md and docs/api.md. Build
backend/app/main.py with FastAPI,
CORS enabled for http://localhost:5173, and
stub versions of every
endpoint in docs/api.md returning hard-
coded example JSON that matches
the documented shapes. Add
backend/app/config.py reading .env and
backend/app/llm.py exposing get_llm(),
which returns a LangChain chat
model pointed at LLM_BASE_URL and
LLM_MODEL. Add a /health endpoint
that calls the LLM with "Reply with OK" and
returns the answer.
Add db.py that creates SQLite tables for
trials, patients, verdicts.
```

Set `.env` to the model infrastructure the organisers give you. If they give a different client than an OpenAI-compatible endpoint, change only `llm.py`.

Done when `uvicorn app.main:app --reload` runs, `/health` returns the model's reply, and every stub endpoint returns JSON.

Phase 3: criteria parser (0:35 to 0:50, P1)

```
Read PHASES.md and docs/spec.md. Write
backend/app/parser.py with
```

```
parse_criteria(trial) -> list[Rule]. Use
get_llm() with a prompt that:
- lists the allowed field names (age, sex,
conditions, medications,
  hba1c, egfr, bmi, systolic_bp, pregnant,
notes) and allowed operators
  (>=, <=, >, <, ==, !=, contains,
not_contains)
- asks for JSON only, one Rule per
criterion, keeping source_text
- maps anything that doesn't fit a field to
field "notes"
Validate the output with Pydantic. If
validation fails, retry once with
the error message included. Cache parsed
rules in SQLite so each trial
is parsed only once. Add a unit test with
one known trial.
```

Done when all 4 trials parse into valid rules and a second call reads from the cache instead of calling the LLM.

Phase 4: rule engine (0:35 to 0:50, P2 in parallel with phase 3)

```
Read PHASES.md and docs/spec.md. Write
backend/app/rules.py with
evaluate(rule, patient) -> CriterionResult
and decide(results) ->
decision, following the decision logic in
```

```
PHASES.md exactly.
- Missing patient value gives UNKNOWN,
  never a guess.
- Normalise units before comparing: glucose
  mmol/L to mg/dL (x18),
  weight lb to kg.
- Case-insensitive matching for conditions
  and medications.
- Rules with field "notes" are not
  evaluated here; return them as
  pending so the explainer phase can send
  them to the LLM.
Write pytest tests for each operator, a
boundary value, a missing
value, a unit conversion, and each branch
of decide().
```

No LLM in this file. That's the point. It's plain Python you can test.

Done when all tests in `tests/test_rules.py` pass.

Phase 5: explainer and screening pipeline (0:50 to 1:05, P1 and P4)

```
Read PHASES.md and docs/spec.md. Write:
1. backend/app/explainer.py
   - check_note_rule(rule, patient) ->
CriterionResult: asks the LLM
   whether the patient's notes satisfy
the rule, answering MET,
```

```
NOT_MET or UNKNOWN as JSON. Handle
negation ("no history of").
Set evaluated_by to "llm".
- explain(verdict, rules, patient) ->
str: a 3 to 5 sentence
    rationale that names each deciding
    criterion and the patient value
    it was checked against. It must not
    contradict the decision.
2. backend/app/screening.py with
screen(trial_id, patient_ids) that
    loads data, gets rules from the parser,
    evaluates them, runs
    note-based rules through
    check_note_rule, calls decide(), computes
    confidence, adds the rationale, saves
    the Verdict, and returns it.
3. Replace the stub POST /screen and GET
    /trials, /patients with real
    implementations.
```

Done when `POST /screen` for one trial returns real verdicts with rationales that match the decisions.

Phase 6: React UI (0:20 to 1:05, P3, in parallel the whole time)

Start at 0:20 against the stub endpoints, so the UI is ready when the real API lands.

```
Read PHASES.md and docs/api.md. In
frontend/, create a Vite + React +
TypeScript app with Tailwind and Recharts.
Put all API calls in
src/api.ts using the base URL
http://localhost:8000. Build:
- Dashboard: list of trials with patient
counts and a "Screen" button
- Screening page: pick a trial, screen all
patients, show a table with
  patient, decision badge (green ELIGIBLE,
  red NOT_ELIGIBLE, amber
  NEEDS_REVIEW) and confidence bar
- Patient detail drawer: every criterion
with its status, the patient
  value, whether rules or the LLM decided
it, and the rationale
- A "Screen all trials" button that shows a
patient x trial grid
Show a loading state during screening and
an error message if the API
fails. Keep it clean and simple.
```

Done when the browser agent can screen a trial and open a patient's detail with real data.

Phase 7: evaluation (1:05 to 1:15, P4)

```
Read PHASES.md. Write
backend/app/evaluate.py that screens every
```

```
patient against every trial, compares with
backend/data/labels.json,
and returns accuracy, precision, recall, a
confusion matrix, and the
list of mismatches with both decisions.
Implement GET /metrics. In the
frontend, add an Accuracy page with the
headline accuracy (marked
green if 85% or more), the confusion
matrix, and a mismatch table.
```

If accuracy is under 85%, read the mismatches.
Usually the fix is in the parser prompt or a unit
conversion, not the rule engine.

Done when the Accuracy page shows a real number.

Checkpoint: required features complete

All of these must be true before starting Part B:

- The React UI screens multiple trials end to end through the real API.
- Every verdict shows per-criterion results and a rationale.
- The Accuracy page shows 85% or more.
- The code is committed.

If this isn't true at 1:15, the whole team fixes it. Part B is optional. A working Part A beats a broken Part B.

Part B: attractive features

Pick in order. Stop at the feature freeze (1:35).

Phase 8: agentic layer (P1, about 15 min)

```
Read PHASES.md. Write backend/app/agent.py:  
a single LangChain  
tool-calling agent with tools  
parse_criteria, evaluate_rules,  
check_note_rule, explain. The agent screens  
one patient against one  
trial by deciding which tools to call.  
After it reaches a decision,  
add a self-verification step: re-run  
decide() on the results and, if  
the agent's decision or rationale  
disagrees, retry once and log the  
correction. Add GET /screen/stream (Server-  
Sent Events) that streams  
each step (tool called, result,  
verification) as JSON. In the  
frontend, add a live "Agent reasoning"  
panel on the screening page  
that shows the steps as they arrive.
```

Keep the phase 5 pipeline as the fallback. If the agent misbehaves in the demo, switch back.

Phase 9: generative AI features (P2 and P1, about 15 min)

Do these in this order and stop when time runs out.

1. LLM-as-judge. A second prompt scores each rationale for clarity from 1 to 5 with a one-line reason. Show the average on the Accuracy page. This measures the "explanation clarity" metric from the problem statement.
2. Natural-language query. Add `POST /query`. The LLM turns a question like "which patients over 60 are eligible for the diabetes trial" into a filter over saved verdicts. Add an "Ask" page with a chat box.
3. RAG with citations. Split each trial's criteria text into chunks in ChromaDB. The explainer retrieves the matching chunk and cites it, for example "Protocol section: inclusion 3".

Phase 10: machine learning features (P4, about 10 min)

1. Anomaly detection. Train an Isolation Forest on patient labs. Flag outliers such as HbA1c 45 on the dashboard and force those patients to NEEDS_REVIEW.
2. Trial ranking. For one patient, embed their profile and all trials, rank trials by similarity plus the rule-

based result, and show a "Best-matching trials" list in the patient drawer.

3. Only with spare time: calibrated confidence with logistic regression trained on the labeled set, or K-Means cohort clusters as a scatter plot.

Phase 11: test hardening (1:35 to 1:45, everyone)

Feature freeze starts here. Only fix bugs.

Read PHASES.md. List 10 edge cases that could break this screener and write pytest tests for them. Include negation in notes, boundary ages, missing labs, mixed units, contradictory notes, an empty trial, a malformed patient record, and nested criteria like "(A or B) and not C". Run them and report which fail.

Fix the failures you can. Write down one failure you found and fixed. The jury asks about this.

Phase 12: package and demo (1:45 to 2:00, everyone)

Read PHASES.md, docs/spec.md and the code. Write README.md covering:

what the app does, how to run backend and frontend, the architecture, where the data comes from (synthetic, generated by backend/data/generate.py), how the AI parts work (parser, rule engine, explainer, agent), and the measured accuracy.

Then record the demo video and rehearse once.

Demo order, about 3 minutes:

1. The problem in one line and the architecture.
2. Screen a trial. Show the agent reasoning panel if phase 8 is done.
3. Open a patient. Show the criteria, the values, and the rationale.
4. Show any Part B features that work: anomaly flag, Ask page, citations.
5. The Accuracy page.
6. docs/spec.md , docs/tasks.md and docs/prompts.md , and the one decision that came from your team rather than the model.

Fallbacks

- Slow model: pre-run screening before the demo and serve saved verdicts.

- Agent breaks: switch the UI back to the phase 5 pipeline.
- Frontend and backend integration breaks: have React read saved verdict JSON.
- Anything else: `git checkout` the last passing phase.